

Laboratory 6

Variant 2, Group 32

By Adam Shahumi and Santiago Puig-Terrero López

1. Introduction

The objective of this assignment is to implement the Q-Learning algorithm from scratch in Python to solve a toy Reinforcement Learning problem. The specific variant assigned is Variant 2, which requires solving the MountainCar-v0 environment from the Gymnasium library. Because the observation space in this environment is continuous, the implementation must include a quantization method to represent the states within a discrete Q-table.

2. Environment Description

Based on the MountainCar-v0 Gymnasium documentation, the environment dynamics are as follows:

- **Goal:** The agent must drive an underpowered car up a steep hill to reach a flag on the right side. The car cannot simply accelerate up the hill; it must build momentum by driving back and forth between the two slopes.
- **Action Space:** The action space is discrete, allowing 3 actions: push left (0), no push (1), and push right (2).
- **State Space:** The observation space is continuous and consists of two variables: the car's position on the track (ranging from roughly -1.2 to 0.6) and its velocity (ranging from -0.07 to 0.07).
- **Rewards:** The agent receives a reward of -1 for every time step taken until it reaches the goal.
- **Terminal Conditions:** An episode terminates either when the car reaches the flag (position ≥ 0.5) or when the maximum episode length of 200 steps is reached.
- **Quantization Method:** Because the state space is continuous, it cannot be directly used as indices in a Q-table. A quantization approach was implemented by dividing the continuous range of both position and velocity into a 20x20 grid. This maps the infinite possible continuous states into $20 \times 20 = 400$ discrete bins, allowing the Q-learning algorithm to function.

3. Experiments and Results

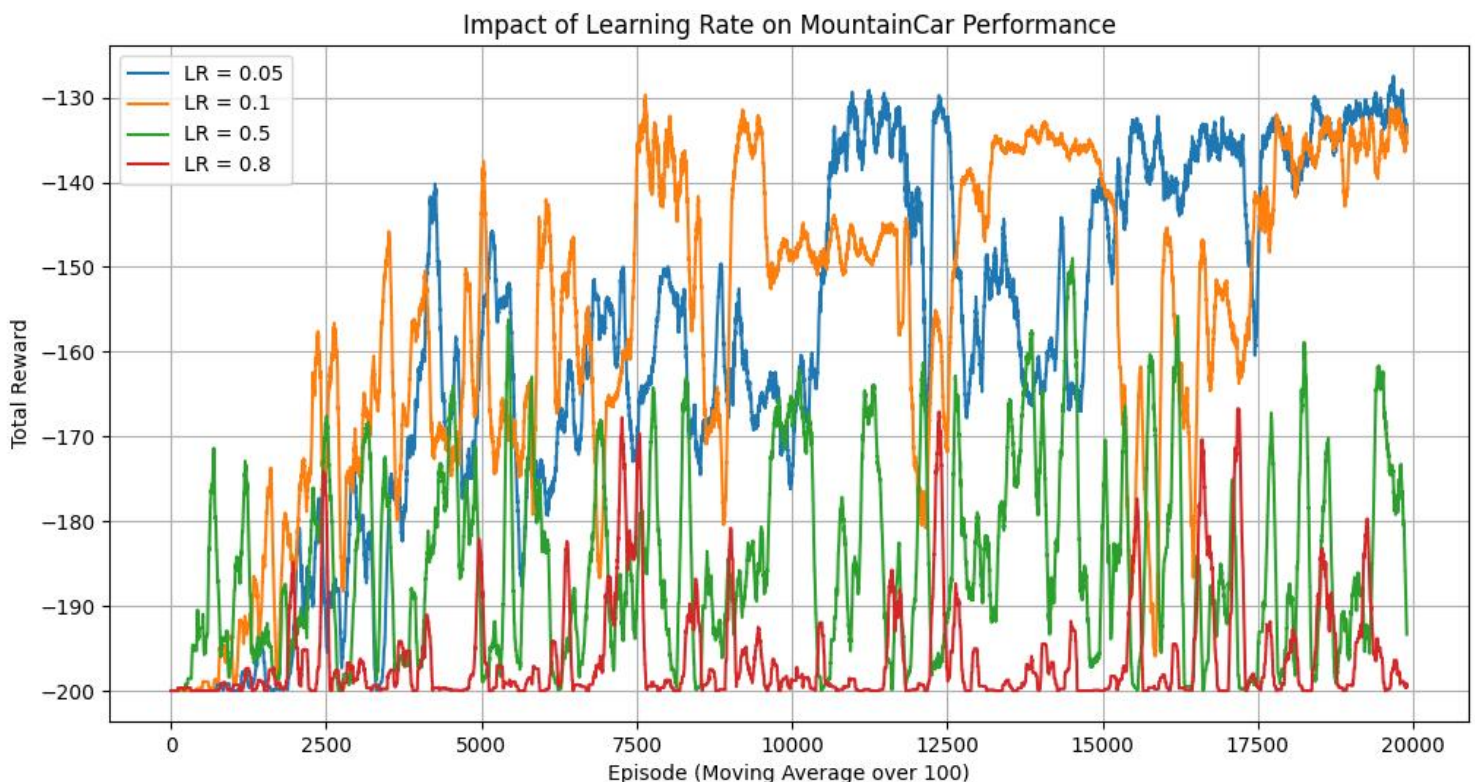
To find the optimal configuration for the environment, the algorithm was tested across three different hyperparameters: Learning Rate (α), Discount Factor (γ), and Exploration Rate Decay (ϵ -decay).

General Experiment Setup:

- **Metric Measured:** The total reward per episode. Because Q-learning in a quantized continuous space is highly stochastic, the results are smoothed using a moving average over 100 episodes to better visualize overall convergence trends.
- **Training Length:** 20,000 episodes per test. This extended horizon ensures the algorithm has ample time to reveal long-term convergence stability rather than just short-term learning speed.
- **Base Hyperparameters:** Learning Rate = 0.1, Discount Factor = 0.99, Epsilon Decay = 0.95. Initial Epsilon was set to 1.0, with a minimum floor of 0.01.
- **State Quantization:** 20 bins for position and 20 bins for velocity.

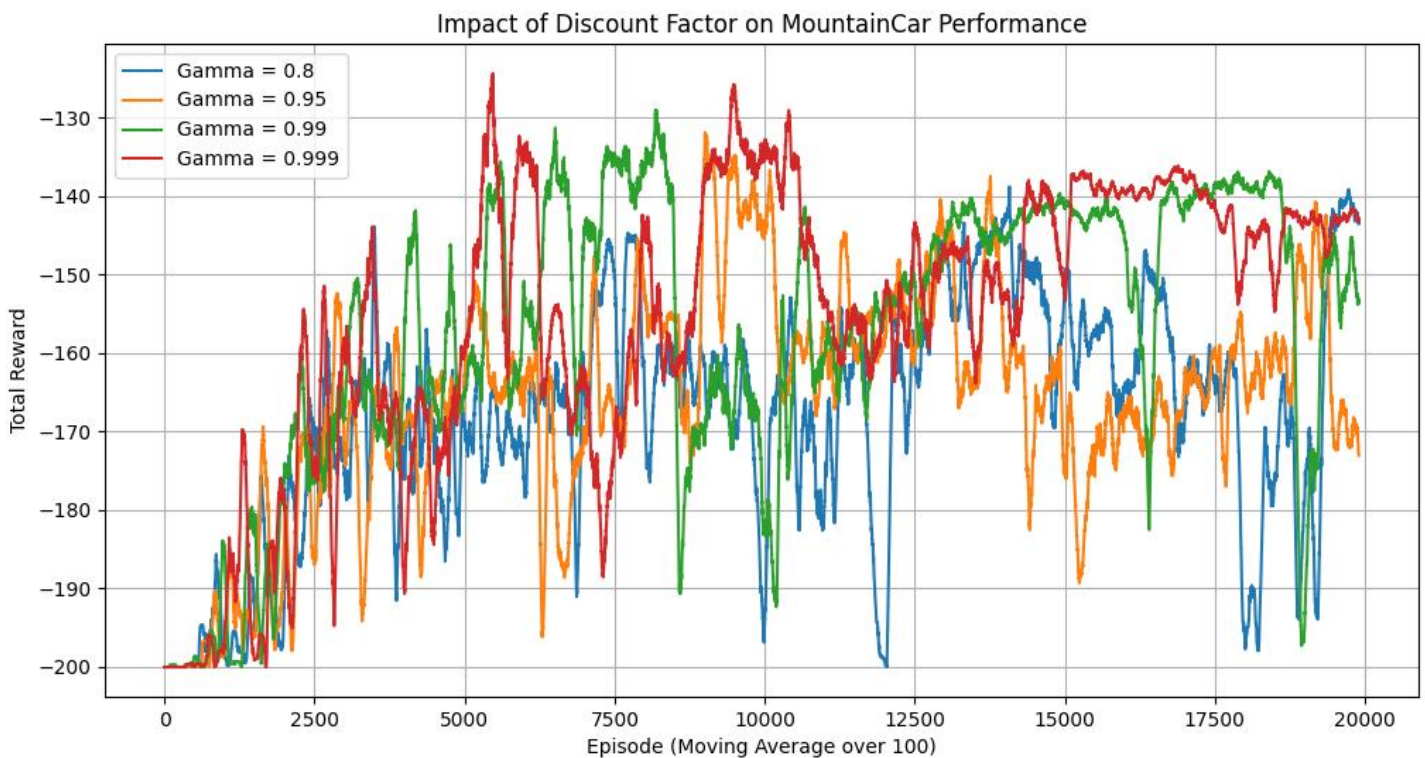
3.1 Experiment 1: Impact of Learning Rate (α)

- **Setup:** Tested values of {0.05, 0.1, 0.5, 0.8}.
- **Results:** The learning rate dictates the extent to which newly acquired information overrides old information in the Q-table. As seen in the figure below, a highly aggressive learning rate ($\alpha=0.8$) is catastrophic. The agent rapidly overwrites its past knowledge based on single unrepresentative transitions, destroying any fragile momentum-building policy. Interestingly, while $\alpha=0.1$ converges faster in the early stages, the extended 20,000-episode horizon reveals that a slower learning rate of $\alpha=0.05$ is ultimately superior. After approximately 12,500 episodes, the 0.05 configuration smoothly overtakes the others, providing the most stable and highest-performing long-term policy, but its really close to the 0.1 value results.



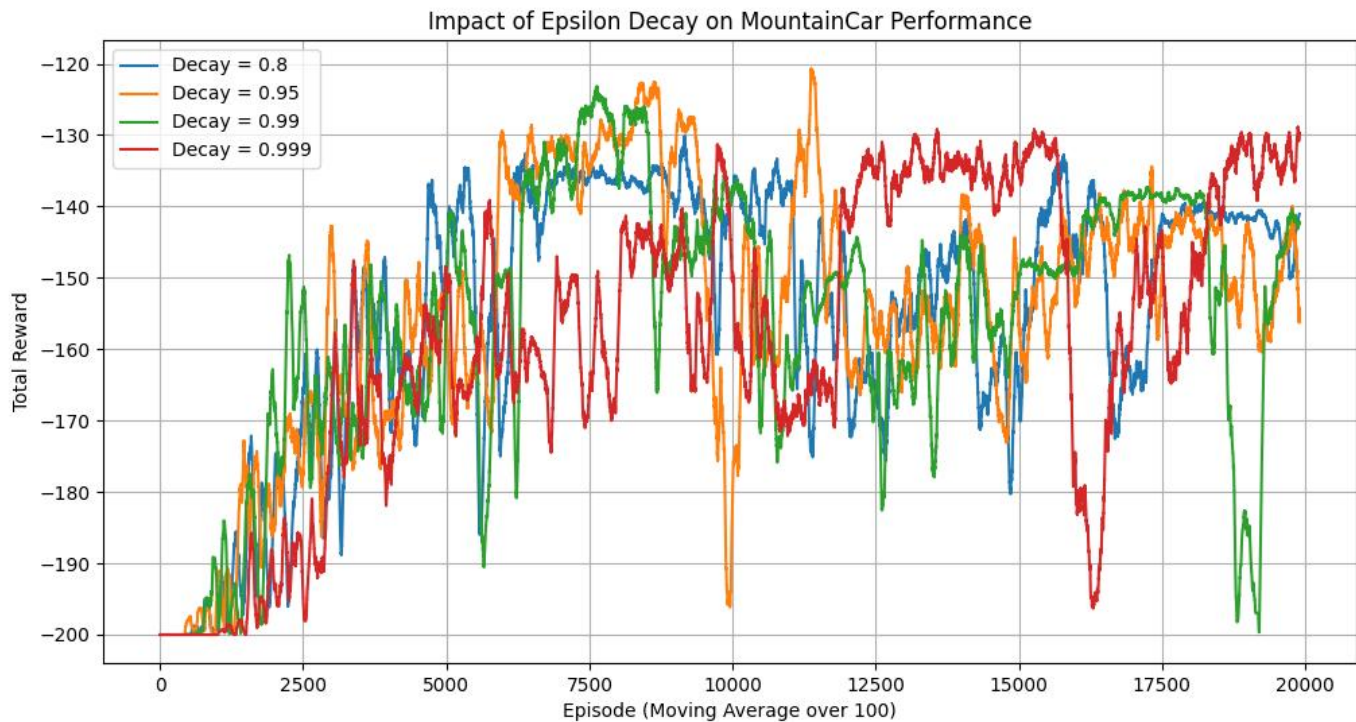
3.2 Experiment 2: Impact of Discount Factor (γ)

- **Setup:** Tested values of $\{0.8, 0.95, 0.99, 0.999\}$.
- **Results:** The discount factor determines how much the agent values future, delayed rewards compared to immediate ones. Because MountainCar utilizes a "penalty for living" reward structure, the agent must plan dozens of steps ahead. The results in Figure 2 illustrate that a lower discount factor ($\gamma=0.8$) makes the agent excessively short-sighted and volatile. Over the 20,000 episodes, extreme long-term planning proves vital. Both $\gamma=0.99$ and $\gamma=0.999$ converge to highly optimal policies, with $\gamma=0.999$ showcasing exceptional long-term stability near the maximum achieved score, successfully propagating the terminal reward backward through the state space.



3.3 Experiment 3: Impact of Epsilon Decay

- **Setup:** Tested values of $\{0.8, 0.95, 0.99, 0.999\}$.
- **Results:** The decay rate governs the exploration-exploitation dilemma. The figure below perfectly demonstrates how training length dictates optimal decay. An excessively fast decay (0.8 or 0.95) forces the agent to stop exploring too early, leading to sub-optimal long-term performance. With an extended horizon of 20,000 episodes, the slowest decay of 0.999 emerges as the definitive optimal choice. It forces the agent to thoroughly map out the state space for over 10,000 episodes. Once the exploration rate finally approaches its minimum, the agent utilizes that vast foundational knowledge to achieve the highest and most robust performance peaks on the graph.

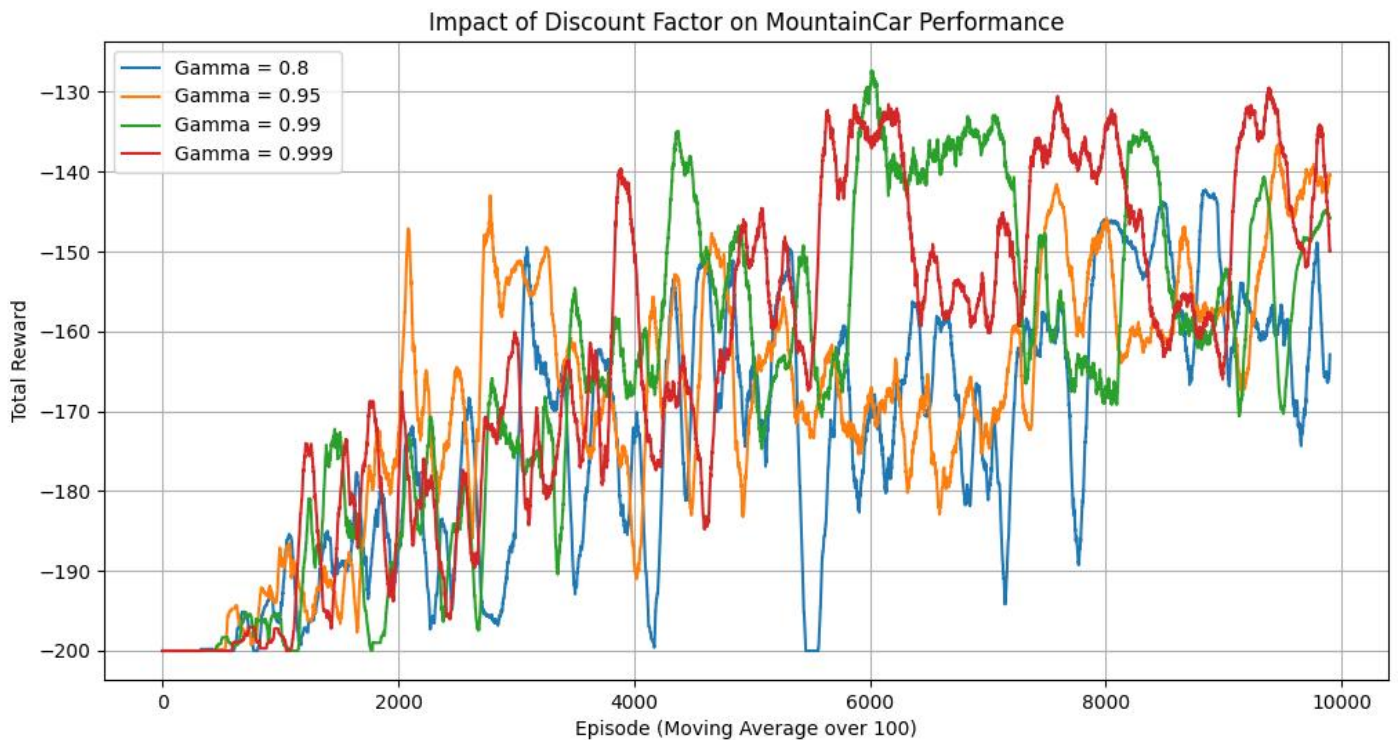
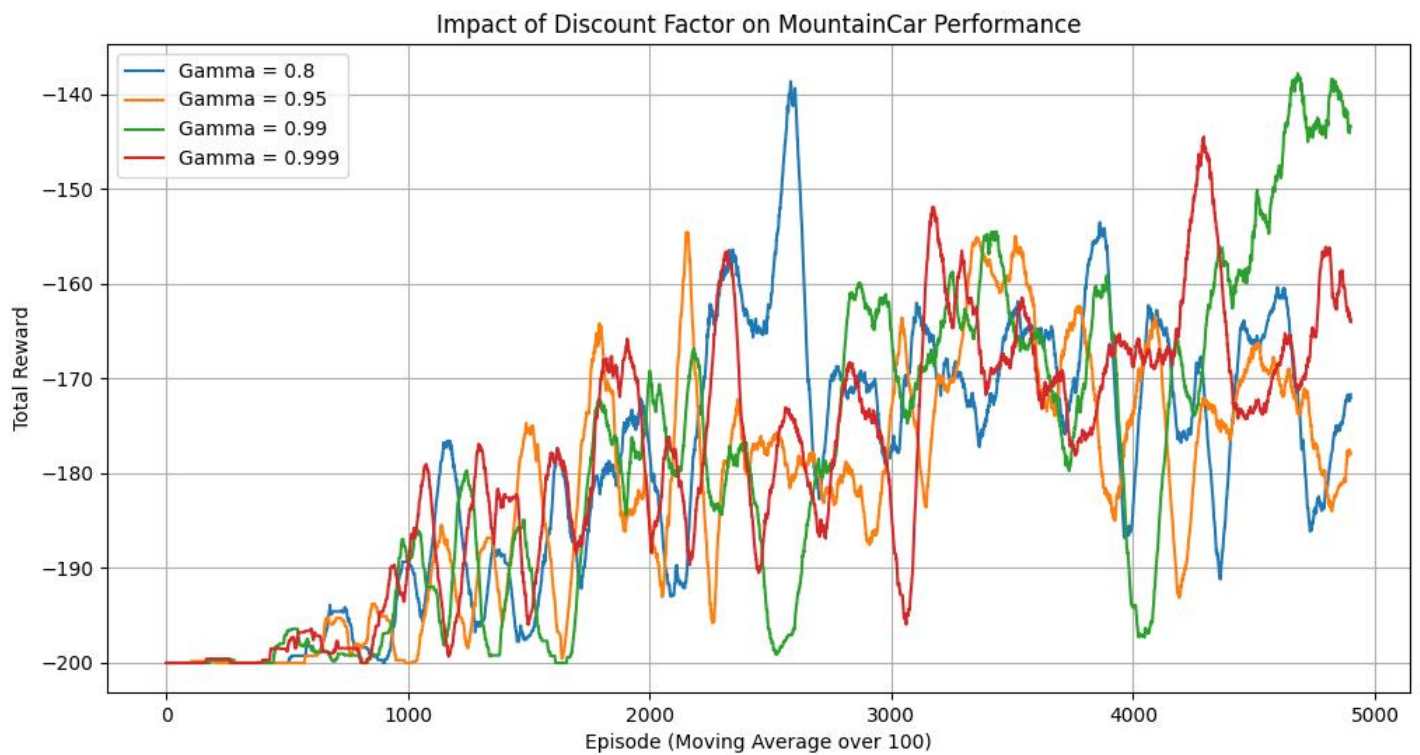


3.4 The Importance of Training Horizon: A Case Study on Discount Factor

To fully understand the true optimal hyperparameters, it became necessary to extend the training horizon significantly. Initial baseline tests were run for 5,000 episodes, which ultimately provided an incomplete picture of convergence. A shorter training window heavily biases the results toward parameters that prioritize rapid, short-term learning over genuine long-term policy stability.

The profound impact of the training horizon is best illustrated by observing the Discount Factor (γ) across different episode limits:

- **At 5,000 Episodes:** As seen in the initial tests, the configuration of $\gamma=0.99$ appears strictly optimal. It climbs rapidly and establishes a strong average reward early on. Meanwhile, extreme long-term planning ($\gamma=0.999$) appears excessively sluggish, learning too slowly to be deemed effective within this short computational window.
- **At 10,000 Episodes:** Doubling the training time reveals a dramatically different narrative. The $\gamma=0.999$ configuration finally completes its prolonged learning phase and converges, catching up to and matching the stability of $\gamma=0.99$.



This phenomenon confirms that in environments with highly delayed rewards like MountainCar-v0, extreme long-term foresight is fundamentally superior, but it demands an extensive amount of time to propagate those distant terminal rewards backward through the continuous state space.

Observing this convergence shift at 10,000 episodes justified the final methodological decision to run all ultimate hyperparameter experiments for 20,000 episodes. Expanding the horizon ensured that the final parameter recommendations ($\alpha=0.05$, $\gamma=0.999$, $\epsilon\text{-decay}=0.999$) represented true algorithmic convergence rather than merely an artifact of an abbreviated training schedule.

4. Conclusion

Through systematic hyperparameter tuning over an extensive 20,000-episode horizon, the Q-learning algorithm successfully solved the MountainCar-v0 environment. The final recommended hyperparameters for this specific environment and a 20x20 quantization grid are $\alpha = 0.05$, $\gamma = 0.999$ and $\epsilon\text{-decay} = 0.999$.

This assignment highlighted that "optimal" hyperparameters are deeply tied to the training horizon. While aggressive parameters work best for short training sessions, solving complex, delayed-reward environments like MountainCar fundamentally rewards patience. It strictly requires extensive exploration ($\epsilon\text{-decay}=0.999$), an extreme prioritization of future rewards ($\gamma=0.999$), and a cautious integration of new knowledge ($\alpha=0.05$) to build stable, momentum-based policies. One thing that could be improved upon is the training with the use of even higher amount of episodes (above 100,000) but due to time constraints for running the code for such long periods of time in order to perform this training, the 20,000 value was decided to be sufficient enough for this exploration task.